Gestión de Infraestructura por Código con Terraform

Despliegue reproducible de *The Store* sobre Kubernetes (kind) con red, registry, entornos y segmentación como código

Redes de Información – TP Especial – Pre-entrega – Grupo N – Integrantes: *Nombre 1, Nombre 2, Nombre 3* – Repositorio: `github.com/<usuario>/t`

1. Problemática y contexto

The Store es una tienda en línea compuesta por cinco microservicios (`ui` y `carts` y `orders` en Java/Spring Boot, `catalog` en Go y `checkout` en Node/NestJS) que en su arquitectura de referencia usan backends distintos: MySQL, DynamoDB, PostgreSQL con RabbitMQ y Redis. La cátedra provee un script `local.sh` que despliega la aplicación en un clúster *kind* (Kubernetes in Docker) de forma **imperativa**: 400 líneas de bash que crean el clúster, compilan imágenes, las copian a los nodos y aplican un YAML monolítico con los cinco servicios en modo *in-memory*.

Ese enfoque tiene los problemas clásicos que la Infraestructura como Código (IaC) viene a resolver: (a) no hay **estado**: el script no sabe qué existe y sólo puede crear o borrar todo; (b) no **converge**: si alguien cambia algo con `kubectl` nadie lo detecta ni lo corrige (*drift*); (c) no hay **entornos**: no puede expresarse "igual que producción pero con una réplica y sin base de datos"; (d) mezcla compilación, red, clúster y aplicación en un único paso no reutilizable; (e) no es revisable ni versionable como un cambio de infraestructura (no hay *plan* que mostrar antes de aplicar).

El objetivo del TP es reemplazar ese script por una definición **declarativa** en Terraform de *toda* la infraestructura necesaria para The Store (red, registry de imágenes, clúster Kubernetes, ingress, capa de datos, microservicios, políticas de red y TLS), organizada en capas y entornos, y demostrar sobre ella las propiedades que justifican la IaC: reproducibilidad, idempotencia, detección y corrección de *drift*, cambios controlados por *plan*, parametrización por entorno y destrucción completa. Restricciones que condicionan el diseño: sin presupuesto cloud (la cátedra lo aclara), presentación remota (todo debe correr en una notebook sin depender de servicios externos) y el requisito de implementar exactamente lo comprometido en este documento.

2. Diseño de la solución

2.1. Principios

- **Todo es código Terraform** (HCL, Terraform $\geq$ 1.10, providers `kreuzwerker/docker`, `tehcyx/kind`, `hashicorp/helm`, `hashicorp/kube` `random`, `tls`, `local`). No se usa el binario `kind` ni scripts imperativos para crear recursos; `kubectl` se usa sólo para inspeccionar y para provocar *drift* en las demos.
- **Tres capas con estado independiente** (*root modules*), comunicadas con `terraform_remote_state`, para acotar el radio de explosión y separar ciclos de vida: (1) plataforma, (2) artefactos, (3) aplicación.
- **Un mismo código, dos entornos** (`dev` y `prod`) mediante *workspaces* de Terraform y archivos `tfvars`, conviviendo en el mismo clúster en namespaces distintos.
- **Versiones fijadas** en todo (providers con lockfile, charts, imagen de nodo por digest, imágenes de datos por versión mayor) para que lo que se presente en noviembre sea lo que se probó en septiembre.

2.2. Capa 1 – Plataforma (`terraform/1-platform`)

Crea la red bridge de Docker `kind` con bloque **172.30.0.0/24** (gateway 172.30.0.1, *masquerade* activado), un registry OCI local (`registry:3`, IP fija 172.30.0.100, publicado sólo en `127.0.0.1:5001`) y el clúster **kind** con Kubernetes **v1.35** (imagen `kindest/node` fijada por digest): un nodo *control-plane* que además actúa como nodo *edge* (publica los puertos 80/443 del host y corre el ingress) y dos nodos *worker*. El clúster se configura con pod CIDR **10.244.0.0/16** (un /24 por nodo), service CIDR **10.96.0.0/16**, API server en `127.0.0.1:6443` y CNI *kindnet* (rutas estáticas entre nodos, soporte de NetworkPolicy). Cada nodo monta un `hosts.toml` generado por Terraform que hace que containerd resuelva `localhost:5001` hacia `kind-registry:5000` (patrón oficial de kind para registries locales). Con Helm se instalan **ingress-nginx** 4.15 (con `hostPort` en el nodo edge) y **metrics-server** 3.14 (Metrics API para el autoscaler).

2.3. Capa 2 – Artefactos (`terraform/2-images`)

Construye las cinco imágenes con el provider de Docker (BuildKit) a partir del código fuente vendorizado y las publica en el registry local. El *tag* de cada imagen es un hash SHA-1 del contenido del código fuente del servicio: si el código no cambia, `terraform plan` está vacío; si cambia un archivo, se reconstruye y publica sólo ese servicio y la capa 3 hace un *rolling update* (artefactos inmutables, sin `latest`). Esta capa es reemplazable por un pipeline de CI/CD sin tocar las otras.

2.4. Capa 3 – Aplicación (`terraform/3-app`, workspaces `dev` y `prod`)

Dos módulos propios reutilizables: `microservice` (ServiceAccount, ConfigMap, Deployment con *hardening* –usuario no root, root filesystem de sólo lectura, sin capabilities, seccomp–, probes, Service ClusterIP, HPA y PodDisruptionBudget opcionales) instanciado cinco veces, y `datastore` (Secret con credenciales generadas por `random_password`, StatefulSet con volumen persistente, Service) instanciado cinco veces en `prod`. Además: namespace por entorno con Pod Security Admission, Ingress por nombre de host (`dev.the-store.localhost`, `the-store.localhost`; `*.localhost` resuelve a loopback por RFC 6761), certificado TLS autofirmado generado por Terraform en `prod`, y **NetworkPolicies** generadas a partir de una tabla de flujos permitidos (modelo *default-deny* + reglas explícitas, equivalente a las ACL de un firewall).

Parámetro	dev	prod
Namespace / host	`the-store-dev` / `dev.the-store.localhost` (HTTP)	`the-store-prod` / `the-store.localhost` (HTTP y HTTPS)
Persistencia	proveedores *in-memory* de cada servicio	MariaDB 11.4 (catalog), PostgreSQL 17 + RabbitMQ 3.13 (orders), Redis 7 (checkout), DynamoDB Local (carts)
Réplicas	1 por servicio	ui 2 (HPA 2–4 al 70 % CPU), catalog 2, resto 1; PDB en los redundantes
Red	sin NetworkPolicies	default-deny ingress+egress + 12 políticas de flujos + DNS + ingress→ui

Tabla 1: Los dos entornos se diferencian únicamente por su archivo `tfvars`.

3. Scope del POC y casos de uso

Incluido (todo lo que se lista se implementa, se prueba con scripts automatizados y se demuestra en vivo):

- **CU-1 Provisionamiento desde cero**: `make up` (equivalente a `terraform apply` en las tres capas) crea red, registry, clúster de 3 nodos, add-ons, imágenes y los entornos `dev` y `prod`; la tienda queda operativa (flujo completo de compra verificado con un *smoke test* de 13 pasos equivalente a los tests e2e de Cypress del proyecto).
- **CU-2 Idempotencia y cambios controlados**: un segundo `terraform plan` no muestra cambios; un cambio de configuración (tema de la UI, réplicas) muestra en el plan exactamente los recursos afectados y se aplica con *rolling update* sin corte de servicio.
- **CU-3 Detección y corrección de *drift***: un `kubectl scale` y el borrado manual de un ConfigMap se detectan en `terraform plan` y se revierten con `terraform apply`.
- **CU-4 Entornos**: mismo código, `dev` e `prod` conviviendo, con nombres de host distintos, persistencia real sólo en `prod` y destrucción independiente de cada uno.
- **CU-5 Segmentación de red como código**: pruebas automáticas de 14 flujos (permitidos y bloqueados, incluido un pod "intruso") sobre las NetworkPolicies; el ingress es el único punto de entrada (DMZ).
- **CU-6 Persistencia y autoscaling**: una orden creada en `prod` sobrevive al reinicio del pod de `orders` (PostgreSQL) y publica un evento en RabbitMQ; el HPA de la UI escala bajo carga (generador Artillery).
- **CU-7 Destrucción completa**: `make down` elimina entornos, clúster, registry y red sin dejar residuos (verificable con `docker ps -a`, `docker network ls`, `docker volume ls`); `make up` vuelve a dejar todo operativo en ≈6 minutos sin reconstruir imágenes.

Excluido (temas de otros grupos o fuera del objetivo): despliegue en nube pública (se explica como evolución del mismo código), observabilidad, logs centralizados, CI/CD, service mesh/WAF, chat con IA de la UI, IPv6, backend remoto de estado (se discute conceptualmente).

4. Diagrama de arquitectura

La Figura 1 (página siguiente) muestra la arquitectura completa. Resumen de los bloques de direccionamiento, sistemas operativos y protocolos:

Segmento	Bloque / direcciones	Función, SO y protocolos
LAN del host	192.168.1.0/24 (host .50, GW .1)	Notebook Ubuntu 24.04 (kernel 7.0), WiFi 802.11ax; salida a Internet por NAT del router/ISP (pull de imágenes públicas y charts).
Red Docker `kind`	172.30.0.0/24, GW 172.30.0.1 (bridge `br-*` del host)	Bridge L2 (un dominio de broadcast, un par veth por contenedor) + NAT: DNAT de puertos publicados (docker-proxy/iptables) y SNAT MASQUERADE de salida. DNS interno de Docker por nombre de contenedor.
Nodos kind	control-plane 172.30.0.2, worker 172.30.0.3, worker2 172.30.0.4 (IPAM dinámico)	Contenedores `kindest/node` (Debian 12, systemd, containerd 2.2, kubelet 1.35). API server HTTPS/mTLS 6443 (publicado en 127.0.0.1:6443); etcd 2379; kube-proxy en modo iptables.
Registry	172.30.0.100, TCP 5000 (HTTP, API OCI v2); host 127.0.0.1:5001	`registry:3`; alias de containerd `localhost:5001` → `kind-registry:5000` en cada nodo.
Pods	10.244.0.0/16: 10.244.0.0/24 (cp), 10.244.1.0/24 y 10.244.2.0/24 (workers)	CNI kindnet: rutas estáticas por nodo (`10.244.1.0/24 via 172.30.0.4`), sin encapsulación. Pods de la app: Amazon Linux 2023 (Java 21 / Go / Node 20), HTTP 8080; datos: imágenes oficiales Debian/Alpine.
Services	10.96.0.0/16; CoreDNS 10.96.0.10 (53 UDP/TCP)	ClusterIP = IP virtual resuelta por DNAT de kube-proxy (balanceo L4). Nombres `<svc>.<ns>.svc.cluster.local`.
Ingress (DMZ)	nodo edge, hostPort 80/443 ← host 0.0.0.0:80/443	ingress-nginx (proxy L7): enruta por cabecera `Host`; HTTP/1.1 y TLS 1.3 con certificado autofirmado (Secret `kubernetes.io/tls`). Único punto de entrada.
Capa de datos (prod)	ClusterIP, sólo alcanzables desde su consumidor	MySQL 3306/TCP (MariaDB), PostgreSQL 5432/TCP, AMQP 0-9-1 5672/TCP (+ consola HTTP 15672), RESP 6379/TCP, DynamoDB HTTP 8000/TCP; volúmenes PVC de 1 GiB (local-path).

Tabla 2: Plan de direccionamiento y protocolos. Los cuatro bloques (LAN, nodos, pods, services) son privados (RFC 1918) y disjuntos entre sí.

Camino de un request (norte-sur): navegador → `the-store.localhost` = 127.0.0.1:443 → DNAT de Docker → 172.30.0.2:443 (hostPort) → ingress-nginx (L7, `Host`) → Service `ui` (ClusterIP :80, DNAT de kube-proxy) → pod `ui` 10.244.x.y:8080 → backends por HTTP 8080 (este-oeste) → datastores. Egreso a Internet: pod → nodo → SNAT del bridge (192.168.1.50) → NAT del ISP (doble/triple NAT).

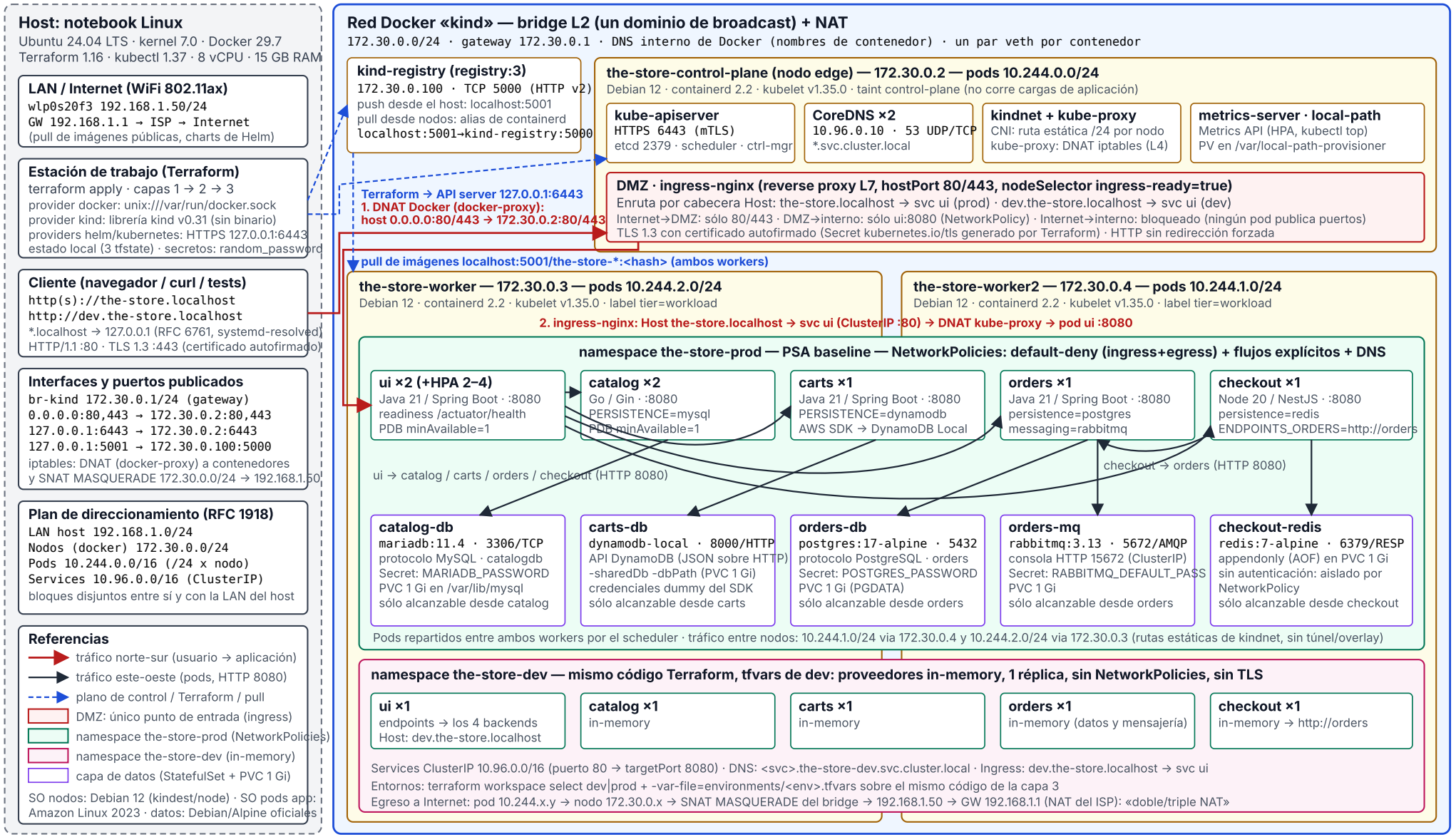


Figura 1: Arquitectura del POC. Rojo: tráfico norte-sur (usuario → aplicación) con sus dos DNAT; negro: tráfico este-oeste entre pods (HTTP 8080) y hacia la capa de datos, que coincide con los flujos permitidos por las NetworkPolicies; azul punteado: plano de control (Terraform hacia Docker y el API server) y pull de imágenes.

5. Alternativas consideradas

Decisión	Elegido y por qué	Alternativas descartadas y motivo
Herramienta de IaC	**Terraform 1.16 (HCL)**: declarativo, *plan*/*apply*, estado explícito, detección de drift, providers maduros para Docker/kind/Helm/Kubernetes.	*Pulumi*: mismo modelo con lenguajes generales, backend por defecto en la nube; *OpenTofu*: compatible (el código corre sin cambios), pero el tema nombra a Terraform; *Ansible*: sin estado ni plan; *Helm/Kustomize solos*: no cubren red, registry ni clúster.
Plataforma	**kind en Docker local**: cero costo, sin dependencias externas en la demo, Kubernetes *vanilla*, misma base que el `local.sh` provisto, toda la red es inspeccionable.	*AWS EKS + RDS + ElastiCache + MQ*: costo diario y credenciales, red opaca; *minikube*: un solo nodo por defecto; *k3d/k3s*: Traefik/ServiceLB embebidos esconden lo que queremos mostrar; *Docker Compose*: sin Kubernetes.
Creación del clúster	**Recurso `kind_cluster` del provider tehcyx/kind** (embebe la librería).	`null_resource` + `local-exec` o invocar `local.sh`: imperativo, sin lectura de estado, destroy frágil.
Estructura del código	**Tres capas con estados separados** + `terraform_remote_state`.	*Un solo root module*: destroy accidental total y limitación de los providers kubernetes/helm cuando el clúster se crea en el mismo módulo; *Terragrunt*: herramienta extra innecesaria para tres capas.
Entornos	**Workspaces + tfvars** en un clúster, un namespace por entorno.	*Dos clústeres*: aislamiento total pero duplica RAM/disco; *un directorio por entorno*: patrón válido, más verboso para dos entornos con backend local.
Red de nodos	**Red Docker creada por Terraform, 172.30.0.0/24**.	Dejar que kind cree su /16 por defecto: bloque no reproducible ni documentado; usar `KIND_EXPERIMENTAL_DOCKER_NETWORK`: depende de una variable de entorno invisible.
CNI / ruteo	**kindnet** (rutas estáticas, sin overlay, NetworkPolicy soportada desde kind 0.24).	*Calico* (BGP entre nodos o VXLAN): atractivo para la materia pero innecesario con todos los nodos en un mismo segmento L2 y más recursos; *Cilium*: ídem, mayor complejidad.
Exposición	**Ingress-nginx con hostPort + port mapping de kind** (DMZ, virtual hosting por `Host`).	*MetalLB L2 + LoadBalancer*: funciona en Linux pero no en macOS/Windows y no aporta a la demo; *NodePort*: puertos altos, sin virtual hosting; `port-forward`: efímero.
Imágenes	**Registry local + alias de containerd**, build en Terraform con tag por hash.	`kind load docker-image`: copia a todos los nodos, imperativo, se pierde al recrear el clúster; *Docker Hub/GHCR*: cuenta e Internet en la demo.
Capa de datos	**StatefulSets propios con imágenes oficiales** (módulo `datastore`).	*Charts de Bitnami*: catálogo gratuito restringido desde 2025; *operadores*: sobredimensionados; *bases fuera del clúster*: dos mundos de red.
Aislamiento	**NetworkPolicies default-deny generadas de una tabla de flujos**.	*Namespaces como aislamiento*: no son VLAN, no filtran tráfico; *políticas L7 (mesh)*: fuera de alcance.
Estado y secretos	**Backend local con locking**, secretos generados y marcados `sensitive`, estado fuera de git.	*Backend remoto (S3/HCP)*: requiere cuenta; se explica en la presentación junto con el manejo de secretos en producción (Vault, cert-manager).

6. Plan de verificación y entregables

Estado actual: el POC descripto está **implementado y probado de punta a punta** en una notebook Ubuntu 24.04 (`make up` desde cero: plataforma en 90 s, imágenes en 12 min la primera vez, `dev` en 1 min y `prod` en 3 min). Cada caso de uso tiene una verificación automatizada y reproducible por la cátedra:

Caso	Comando	Criterio de aceptación
CU-1	`make up` + `make smoke ENV=prod`	3 nodos `Ready`; smoke test 13/13 (home, catálogo de 6 productos, detalle, carrito, checkout con totales $150 + $10 + $5 = $165) en `dev` (HTTP) y `prod` (HTTP y HTTPS).
CU-2	`make plan ENV=prod`; `terraform apply -var ui_theme=orange`	Segundo plan sin cambios; el cambio de tema muestra exactamente 2 recursos (ConfigMap y Deployment) y se aplica con *rolling update* en <30 s.
CU-3	`make drift-demo ENV=prod`	El plan muestra `replicas "3> "2"` y la recreación del ConfigMap borrado; el apply los corrige.
CU-4	`make app ENV=dev`, `make destroy-app ENV=dev`	`dev` y `prod` operativos a la vez con hosts distintos; destruir `dev` no afecta a `prod`.
CU-5	`make netpol-test ENV=prod`	14/14: flujos declarados abiertos; intruso y flujos no declarados bloqueados (timeout).
CU-6	`make persistence-test ENV=prod`; `make load ENV=prod`	La orden creada sigue tras reiniciar `orders`; cola `orders-orders-queue` con mensajes; HPA de `ui` escala de 2 a 4 réplicas bajo carga.
CU-7	`make down`; `make up`	Sin contenedores, redes ni volúmenes remanentes; la recreación completa (sin reconstruir imágenes) tarda $\approx$6 min.

Entregables: código Terraform con módulos, `Makefile` y scripts de verificación; documento *how-to* paso a paso (README del repositorio); registro de decisiones de diseño con alternativas descartadas; presentación (PPT) con demostración en vivo de CU-1 a CU-7 y video de respaldo. **Cronograma**: 22/09 pre-entrega; 24/09–15/10 correcciones de la revisión, pruebas e2e oficiales (Cypress) y pulido de la documentación; 16/10–31/10 presentación, ensayos cronometrados y video; 04/11 entrega final.